01_CODE_ALONG_LAB

Pintop | 2026 Training

CLASSROOM CODE-ALONG - Unit-Integration Testing & OpenAPI (Backend Design & Engineering)

Pintop Technologies | Class 11 | Instructor and student companion

What we will produce

Recreate a documented excerpt from the **real project** (`tests/Feature/PayTrackTest.php`), practise a small safe variation, verify using the complete app or appropriate tool, and record both success and failure evidence. Preserve the original project.

Why: You are asked to change one function in a large codebase. Without tests you cannot be sure your change did not break something else. Today we build a safety net and give the API a written description other teams can rely on.

1. Find and read the real file

`codebase/tests/Feature/PayTrackTest.php`

The excerpt is in `REFERENCE EXCERPT.php`. Its job is to give you something to type, trace and understand. It is not guaranteed to run on its own: it is part of a larger file. Do not overwrite project source with a fragment.

2. Set up the real project

```
cd "Backend Design & Engineering/codebase"
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan serve --port=8000
```

3. Type in stages

Stage 1

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
class PayTrackTest extends TestCase {
 use RefreshDatabase;
 public function test_health():void{$this->getJson('/health/live')->assertOk()->assertJsonPath('status','al
```

Stage 2

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
 public function test_customer_registers_and_has_wallet():void{
  $this->postJson('/v1/auth/register',['name'=>'Tola Ade','email'=>'tola@example.test','password'=>'veryGoo
  $this->assertDatabaseHas('wallets',['balance_minor'=>0]);
```

Stage 3

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
        }
        public function test_transfer_requires_authorisation():void{$this->postJson('/v1/transfers',[])->assertUna
       }
```

4. Complete these activities

1. **Read the existing test name and identify the condition it is protecting.**
2. **Write a new test for one boundary case in the same style; deliberately see it fail before the fix.**
3. **Make the smallest local code or fixture change that passes the test; rerun the suite and record evidence.**

5. Check, debug and record

```
php artisan route:list --path=api
php artisan test
```

Expected check: If the local app starts or its tests pass, capture evidence of the specific behaviour tied to this class, not merely an application startup. Test one failed or edge input too. Log the exact error if blocked.

6. Marking

- 4/20: explain concept and input/output in plain language
- 4/20: accurate typed or edited work
- 4/20: local proof and explanation
- 4/20: safe variation and wrong-case recovery
- 4/20: short documented submission

Lab ethics: no unapproved public scanning, no third-party tokens and no real banking customer data.